Universidad del Valle de Guatemala
Modelación y simulación

Laboratorio 5: Algoritmo genético para el TSP

Erick Guerra
Fabián Morales
Diego Patzán

1. Introducción

Este informe describe la implementación de un algoritmo genético para el problema del agente viajero. El algoritmo se aplicó a la instancia Berlin52 de TSPLIB y a una configuración propia de nodos. Se presentan sus componentes, los parámetros utilizados y la evolución de la población hasta la mejor ruta encontrada.

2. Representación y función de fitness

Cada individuo es una permutación de las n ciudades. El orden de la permutación indica la secuencia de visita y el recorrido regresa a la primera ciudad al terminar. Con esta codificación toda solución visita cada ciudad exactamente una vez, por lo que nunca se generan rutas inválidas.

La función de fitness es la distancia total del recorrido y se busca minimizarla:

$$D = d_{\pi 1\, \pi 2} + d_{\pi 2\, \pi 3} + \ldots + d_{\pi n\, \pi 1}$$

donde π_i es la ciudad en la posición i y d_{ij} es la distancia entre las ciudades i y j. La entrada puede ser un listado de coordenadas x, y, un archivo en formato TSPLIB o una matriz simétrica de distancias. Con coordenadas se construye la matriz euclidiana y en Berlin52 se redondea al entero más cercano, como indica TSPLIB. Si solo se recibe la matriz, se reconstruyen coordenadas aproximadas con escalamiento multidimensional clásico para poder dibujar las rutas. La población completa se evalúa en una sola operación de NumPy.

Código 1. Evaluación de la población

```
def distancia_rutas(poblacion, matriz):
    siguientes = np.roll(poblacion, -1, axis=1)
    return matriz[poblacion, siguientes].sum(axis=1)
```

3. Población inicial

La población inicial está formada por N permutaciones aleatorias con distribución uniforme. No se usan heurísticas de construcción, así que toda la mejora proviene de los operadores genéticos. En Berlin52 la mejor ruta inicial midió 24108, más de tres veces el óptimo conocido.

4. Métodos de selección

Se implementaron cuatro esquemas. El mismo esquema se usa para elegir a los padres y a los sobrevivientes. Con elitismo los sobrevivientes son directamente los mejores. Con los otros esquemas se eligen de forma aleatoria y siempre se conserva al mejor individuo.

Tabla 1. Esquemas de selección implementados

Esquema	Mecanismo	Parámetros
Torneo	Se eligen k individuos al azar y gana el de menor distancia.	k
Ranking	Se ordena la población. El individuo en la posición r recibe peso $N - r$ en la versión lineal o $e^{-pr/N}$ en la versión exponencial.	f, p
Fitness	El peso es $1 / D^p$ en la versión inversa o $D_{máx} - D$ en la versión ventana.	f, p
Elitismo	Los padres se eligen de forma uniforme entre los s mejores.	s

En la tabla 1, p representa la presión de la distribución y $D_{máx}$ la mayor distancia de la población actual. El torneo se implementó de forma vectorizada:

Código 2. Selección por torneo

```
competidores = rng.integers(0, n, size=[cantidad, parametros["k"]])
ganadores = np.argmin(distancias[competidores], axis=1)
return competidores[np.arange(cantidad), ganadores]
```

5. Operadores de cruce y mutación

El cruce de orden copia un segmento aleatorio del primer padre en la misma posición del hijo. Las posiciones restantes se llenan con las ciudades del segundo padre en el orden en que aparecen, sin repetir las del segmento. El hijo siempre es una permutación válida y conserva el orden relativo de ambos padres.

Código 3. Cruce de orden

```
def cruce_ox(padre, madre, rng):
    inicio, fin = np.sort(rng.choice(len(padre), 2, replace=False))
    segmento = padre[inicio:fin]
    resto = madre[~np.isin(madre, segmento)]
    return np.concatenate([resto[:inicio], segmento, resto[inicio:]])
```

Tabla 2. Ejemplo de cruce de orden

Elemento	Valor
Padre 1	1 2 3 4 5 6 7 8
Padre 2	3 7 5 1 6 8 2 4
Segmento del padre 1	posiciones 4 a 6: 4 5 6
Ciudades restantes del padre 2	3 7 1 8 2
Hijo	3 7 1 4 5 6 8 2

La mutación toma un padre seleccionado y le aplica uno de tres operadores: inversión de un segmento con probabilidad 0.6, intercambio de dos ciudades con probabilidad 0.2 e inserción de una ciudad en otra posición con probabilidad 0.2. La inversión tiene mayor probabilidad porque solo cambia dos aristas del recorrido y es la más efectiva para eliminar cruces.

Código 4. Operadores de mutación

```
operador = rng.choice(["inversion", "intercambio", "insercion"], p=[0.6, 0.2, 0.2])
if operador == "inversion":
    hijo[i:j + 1] = hijo[i:j + 1][::-1]
elif operador == "intercambio":
    hijo[i], hijo[j] = hijo[j], hijo[i]
else:
    hijo = np.insert(np.delete(hijo, j), i, hijo[j])
```

Como variante opcional se implementó una búsqueda local 2-opt que se aplica al mejor individuo cada vez que mejora. En los resultados esta variante se llama híbrida.

6. Construcción de cada generación y diversidad

Cada generación nueva se forma con cuatro grupos: s por N sobrevivientes, c por N hijos por cruce, m por N hijos por mutación y el resto con inmigrantes, que son rutas aleatorias nuevas. Los individuos repetidos se detectan por su distancia y se reemplazan por mutaciones de otros miembros de la población. La diversidad se mide como la proporción de individuos distintos en cada generación.

En las ejecuciones reportadas s + c + m = 1, por lo que no se agregaron inmigrantes. La diversidad se mantuvo con el reemplazo de duplicados y con la selección por torneo, que permite que individuos que no son los mejores también tengan descendencia.

7. Parámetros

Los valores entre 0 y 1 se interpretan como porcentaje de N y los valores mayores o iguales a 1 como cantidad de individuos. La tabla 3 resume los parámetros utilizados.

Tabla 3. Parámetros del algoritmo

Parámetro	Descripción	Berlin52	Retícula
N	Tamaño de la población	300	500
maxIter	Número máximo de iteraciones	3000	1000
seleccion	Esquema de selección	Torneo	Torneo
k	Tamaño del torneo	3	3
s	Porcentaje de sobrevivientes	0.1	0.1
c	Porcentaje creado por cruce	0.5	0.5
m	Porcentaje creado por mutación	0.4	0.4
f, p	Distribución y presión para ranking o fitness	No aplica	No aplica
semilla	Semilla del generador aleatorio	1	1

8. Resultados en Berlin52

Con los parámetros de la tabla 3, el algoritmo genético redujo la mejor distancia de 24108 a 7715, a 2.29 % del óptimo conocido de 7542. La figura 1 muestra cómo la ruta pierde cruces a medida que avanzan las iteraciones. La última mejora ocurrió en la iteración 576.

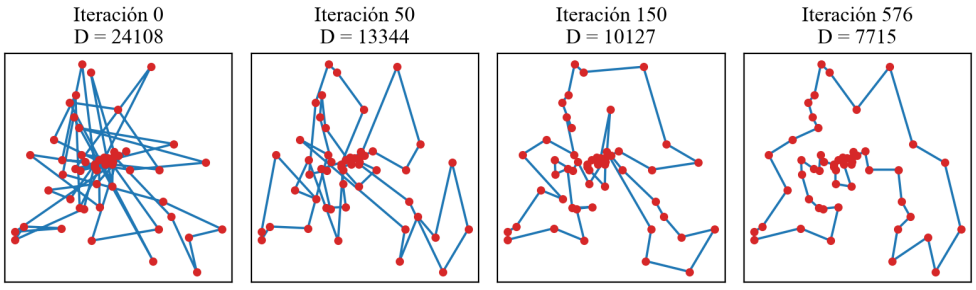


Figura 1. Evolución de la mejor ruta

La figura 2 muestra la minimización del fitness. La mejor distancia cae rápido en las primeras 200 iteraciones y luego se estabiliza. La proporción de individuos distintos se mantuvo entre 0.81 y 1, con un promedio de 0.89, por lo que la población no perdió su diversidad.

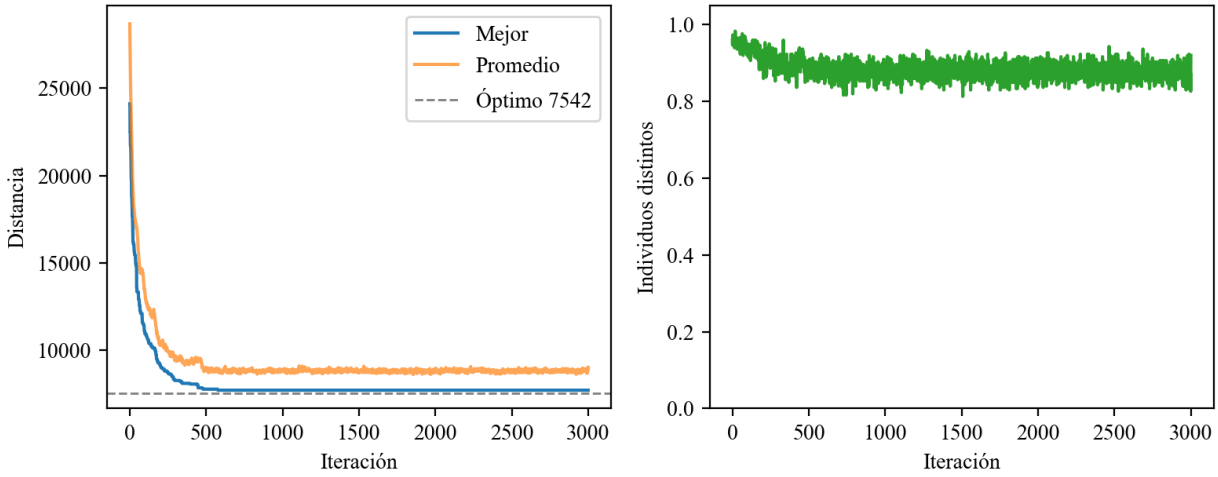


Figura 2. Distancia y diversidad por iteración

Para comparar los esquemas de selección se repitió la ejecución con cada uno, usando 1500 iteraciones y la misma semilla. Los resultados están en la tabla 4 y la figura 3.

Tabla 4. Resultados por esquema de selección

Esquema	Mejor D	Error	Diversidad promedio
Torneo k = 3	7715	2.29 %	0.89
Ranking exponencial	8203	8.76 %	0.82
Fitness ventana	8946	18.62 %	0.97
Elitismo s = 0.1	7994	5.99 %	0.65

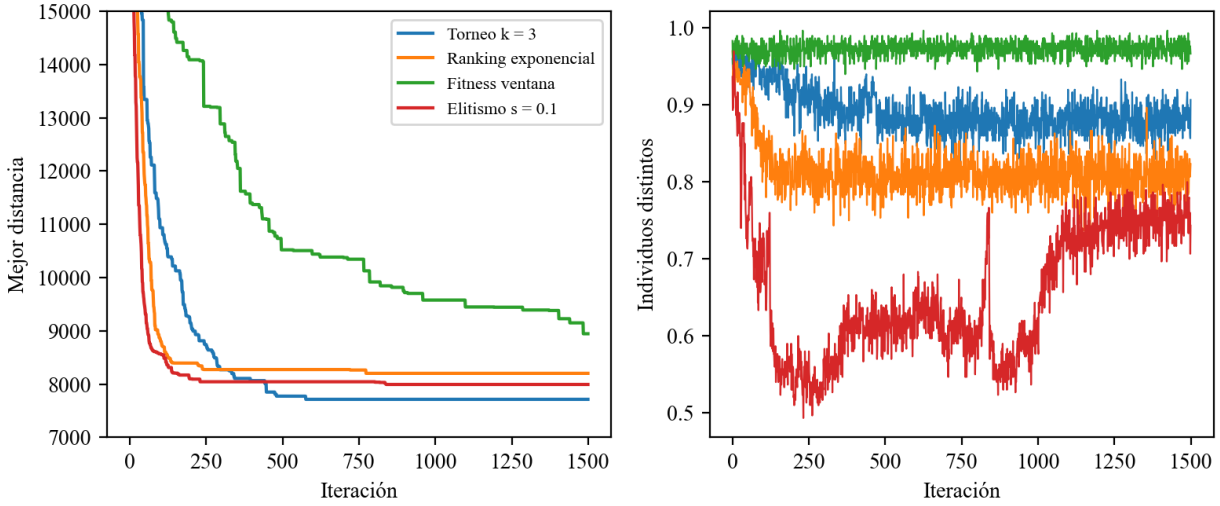


Figura 3. Comparación de esquemas de selección

El elitismo converge más rápido pero es el que más diversidad pierde. Fitness con ventana conserva casi toda la diversidad, pero su presión selectiva es baja y avanza lento. El torneo logra el mejor equilibrio entre ambos efectos y obtiene la menor distancia.

Finalmente se activó la búsqueda local 2-opt con los mismos parámetros. La tabla 5 compara ambas variantes y la figura 4 muestra las rutas finales.

Tabla 5. Resultados en Berlin52

Variante	D	Error	Tiempo
Genético puro	7715	2.29 %	22 s
Genético híbrido con 2-opt	7542	0.00 %	23 s
Óptimo de TSPLIB	7542	0.00 %	No aplica

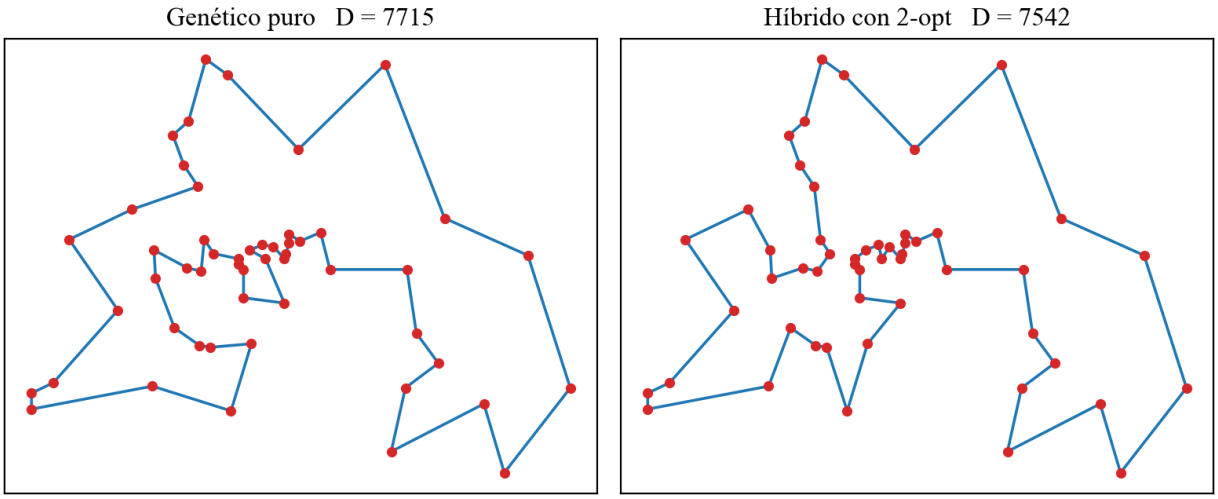


Figura 4. Rutas finales en Berlin52

La mejor ruta encontrada tiene distancia D = 7542, igual al óptimo, y recorre las ciudades en este orden: 1, 22, 31, 18, 3, 17, 21, 42, 7, 2, 30, 23, 20, 50, 29, 16, 46, 44, 34, 35, 36, 39, 40, 37, 38, 48, 24, 5, 15, 6, 4, 25, 12, 28, 27, 26, 47, 13, 14, 52, 11, 51, 33, 43, 10, 9, 8, 41, 19, 45, 32, 49, 1.

9. Resultados en la configuración propia

Se diseñó una retícula de 6 por 6 ciudades separadas 10 unidades. Todo recorrido tiene 36 aristas y cada una mide al menos 10, por lo que ninguna ruta puede medir menos de 360. Como la retícula tiene un número par de ciudades por lado, existe un recorrido que solo usa aristas de longitud 10. Por eso el óptimo es exactamente 360 y sirve para verificar si el algoritmo llega a la solución exacta.

Con los parámetros de la tabla 3, el algoritmo encontró un recorrido de longitud 360 en la iteración 272, como muestra la figura 5.

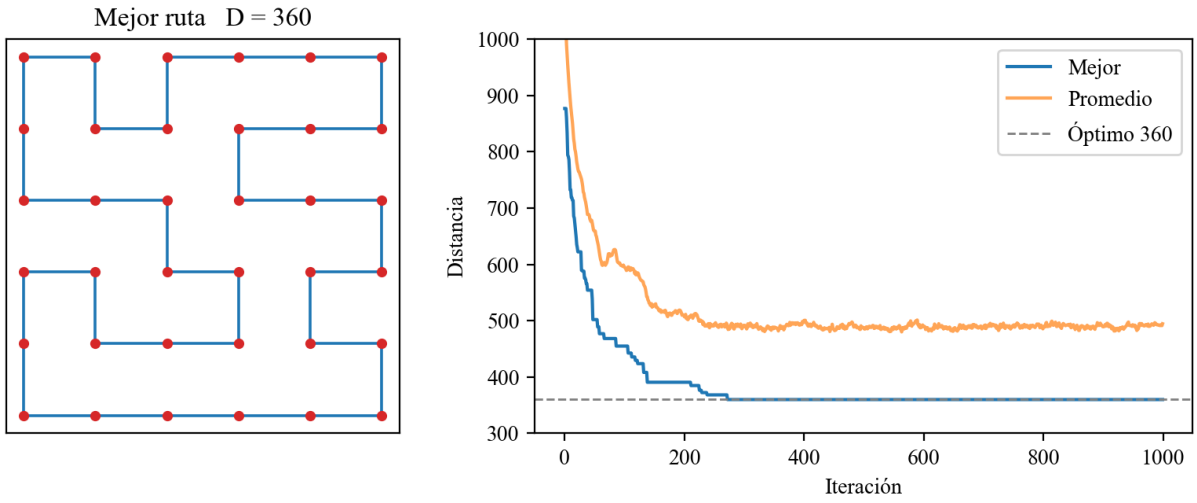


Figura 5. Resultado en la retícula

Tabla 6. Resultados en la retícula

Semilla	Genético puro	Genético híbrido
1	360.00	360.00
2	368.28	360.00
3	368.28	360.00

Con otras semillas el genético puro terminó en 368.28, un recorrido con dos aristas diagonales. El híbrido alcanzó el óptimo en las tres semillas.

10. Animación de la evolución

Junto a este informe se adjunta el archivo evolucion_berlin52.gif. Muestra la evolución del mejor recorrido en Berlin52 y la reducción de su distancia en cada iteración. El notebook Lab5.ipynb contiene la misma animación de forma interactiva.

11. Repositorio

https://github.com/FabianMoraeles/ModSim-lab5